

Classic ML model telemetry to Unity Catalog

This document explains how to get telemetry from a classic machine learning model into Unity Catalog Delta tables on Databricks, with examples for training, batch inference, and real-time serving.

Summary

For classic ML workloads, there are two main patterns:

- For training jobs, notebooks, and batch inference, add tracing in Python code and route the traces to Unity Catalog using MLflow tracing APIs or environment variables.
- For real-time inference on Databricks Custom Model Serving, enable endpoint telemetry on the serving endpoint. Basic Python logging is captured automatically, while custom spans and metrics require OpenTelemetry SDK instrumentation in the served model code.

Requirements

Before implementing either pattern, the workspace and destination must meet these requirements:

- The workspace **must** be enabled for Unity Catalog.
- The destination **must** use managed Delta tables; external storage is not supported for endpoint telemetry.
- The table location **must** be in the same region as the workspace.
- The identity writing data needs Unity Catalog privileges such as `USE CATALOG`, `USE SCHEMA`, and `MODIFY` and `SELECT` on the target tables. `ALL_PRIVILEGES` is not sufficient for the MLflow trace-table path.

1: Training and batch inference with MLflow tracing

This is the simplest pattern for classic ML code that runs in notebooks, jobs, or scheduled batch inference. The key steps are:

- Set the trace destination to a Unity Catalog catalog and schema.
- Instrument functions with `@mlflow.trace` or use `mlflow.start_span(...)` for more control.

Example: traced sklearn training and batch prediction

```
Python
import mlflow
from mlflow.entities import UCSchemaLocation
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

mlflow.set_tracking_uri("databricks")

# Route traces to Unity Catalog
mlflow.tracing.set_destination(
    destination=UCSchemaLocation(
        catalog_name="main",
        schema_name="ml_observability",
    )
)

@mlflow.trace
def prepare_features(df):
    # Example placeholder for feature prep
    return df

@mlflow.trace
def train_model(X_train, y_train):
    model = RandomForestClassifier(
        n_estimators=100,
        max_depth=8,
        random_state=42,
    )
    model.fit(X_train, y_train)
    return model

@mlflow.trace
def batch_predict(model, X_test):
    return model.predict(X_test)

@mlflow.trace
def evaluate_predictions(y_true, y_pred):
    return {
        "accuracy": accuracy_score(y_true, y_pred),
    }

# Example flow
X_train_prepared = prepare_features(X_train)
X_test_prepared = prepare_features(X_test)

model = train_model(X_train_prepared, y_train)
```

```
preds = batch_predict(model, X_test_prepared)
metrics = evaluate_predictions(y_test, preds)

print(metrics)
```

Alternative: use an environment variable instead of the Python API

```
Python
import os
import mlflow

mlflow.set_tracking_uri("databricks")
os.environ["MLFLOW_TRACING_DESTINATION"] = "main.ml_observability"

@mlflow.trace
def train_and_score(X_train, y_train, X_test):
    model = ...
    model.fit(X_train, y_train)
    return model.predict(X_test)
```

When to use this pattern

Use this pattern when the classic ML model runs as:

- a notebook experiment
- a scheduled training job
- batch scoring code
- an offline validation or evaluation pipeline

In those cases, instrumentation lives directly in the Python code and does not require Model Serving endpoint configuration.

2: Real-time inference on Custom Model Serving

For a classic ML model deployed to Databricks Custom Model Serving, there is an additional serving-side step: enable endpoint telemetry on the endpoint.

When endpoint telemetry is enabled:

- Databricks can persist logs, traces, and metrics from the endpoint into Unity Catalog tables.
- In the UI or API, you choose the Unity Catalog location and optional table prefix.

- Databricks writes to tables named `<prefix>_otel_logs`, `<prefix>_otel_spans`, and `<prefix>_otel_metrics`.

Example: basic endpoint logging for a classic ML model

If the goal is to capture inference-time logs, standard Python logging is enough. No OpenTelemetry SDK code is required for basic logging.

```
Python
import logging
import mlflow

class FraudModel(mlflow.pyfunc.PythonModel):
    def load_context(self, context):
        # Replace with your actual deserialization logic
        self.model = ...

    def predict(self, context, model_input):
        logging.warning("Received inference request")
        try:
            preds = self.model.predict(model_input)
            logging.warning("Inference completed successfully")
            return preds
        except Exception as e:
            logging.error(f"Inference failed: {e}")
            raise
```

Example: custom spans and metrics inside served model code

If the goal is richer inference-time observability, add OpenTelemetry SDK instrumentation in the model code. Databricks documents using Open Telemetry Protocol exporters, tracer providers, span processors, and metric readers for this pattern.

```
Python
import os
import mlflow
from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter
from opentelemetry.exporter.otlp.proto.http.metric_exporter import OTLPMetricExporter
from opentelemetry.sdk.resources import Resource
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.sdk.metrics import MeterProvider
from opentelemetry.sdk.metrics.export import PeriodicExportingMetricReader
from opentelemetry.trace import get_tracer, set_tracer_provider
from opentelemetry.metrics import get_meter, set_meter_provider
```

```

resource = Resource.create({
    "service.name": "classic-ml-fraud-model",
    "worker.pid": str(os.getpid()),
})

trace_exporter = OTLPSpanExporter()
tracer_provider = TracerProvider(resource=resource)
tracer_provider.add_span_processor(BatchSpanProcessor(trace_exporter))
set_tracer_provider(tracer_provider)
tracer = get_tracer(__name__)

metric_exporter = OTLPMetricExporter()
metric_reader = PeriodicExportingMetricReader(metric_exporter)
meter_provider = MeterProvider(metric_readers=[metric_reader], resource=resource)
set_meter_provider(meter_provider)
meter = get_meter(__name__)
prediction_counter = meter.create_counter(
    name="prediction_count",
    description="Number of predictions made",
    unit="1",
)

class FraudModel(mlflow.pyfunc.PythonModel):
    def load_context(self, context):
        self.model = ...

    def predict(self, context, model_input):
        with tracer.start_as_current_span("FraudModel.predict") as span:
            span.set_attribute("row_count", len(model_input))
            span.set_attribute("column_count", len(model_input.columns))
            prediction_counter.add(1)

            preds = self.model.predict(model_input)
            return preds

```

Enabling endpoint telemetry

This is the serving-side configuration that complements the code above. It can be done in the UI or API.

UI flow

- Go to Serving.
- Create or edit the serving endpoint.
- In the endpoint telemetry section, select the destination catalog and schema.

- Optionally set a table prefix.

API example

JSON

```
{
  "name": "classic-ml-fraud-endpoint",
  "config": {
    "served_entities": [
      {
        "name": "fraud-model",
        "entity_name": "fraud-model",
        "entity_version": "1",
        "workload_size": "Small",
        "scale_to_zero_enabled": true
      }
    ]
  },
  "telemetry_config": {
    "table_names": {
      "logs_table": "main.ml_observability.fraud_endpoint_logs",
      "metrics_table": "main.ml_observability.fraud_endpoint_metrics",
      "traces_table": "main.ml_observability.fraud_endpoint_spans"
    }
  }
}
```

Which pattern to recommend

A practical recommendation for most classic ML customers is:

- For training and batch scoring, start with MLflow tracing in code because it is lightweight and easy to add around training, prediction, and evaluation steps.
- For real-time serving, first enable endpoint telemetry and add basic Python logging. Only add full OpenTelemetry spans and metrics when they need deeper production debugging or custom application telemetry.

Notes and limitations

- For endpoint telemetry, schema evolution on the target table is not supported.
- Only managed Delta tables are supported.
- External storage and Arclight default storage are not supported for endpoint telemetry.
- The table location must be in the same region as the workspace.
- Log lines must be less than 1 MB, records less than 10 MB, and requests less than 30 MB.

- Enabling tracing on a serving endpoint can reduce serving throughput.
-

Sources

- [patterns-trace-ingestion.md](#)
- [Persist custom model serving data to Unity Catalog | Databricks on AWS](#)
- [Store OpenTelemetry traces in Unity Catalog | Databricks on AWS](#)